Lab 05. Triển khai và đánh giá suy luận Tiny CNN V5 INT8 kết hợp ARM + FPGA trên PYNQ-Z2

Step 1. Mục tiêu và cấu trúc hệ thống

Mục tiêu của Lab 05 là triển khai mô hình Tiny CNN V5 INT8 trên board PYNQ-Z2 theo hai phương án: toàn bộ CNN chạy bằng ARM Cortex-A9 và hệ kết hợp **ARM + FPGA**. Trong cấu hình kết hợp, FPGA thực hiện Conv1 + ReLU + MaxPooling1; ARM tiếp tục xử lý các lớp còn lại để phân loại ảnh MRI não thành bốn lớp.

Hệ thống sử dụng **conv1_accel.bit** để cấu hình FPGA, khối **axi_dma_0** để truyền tensor INT8 và IP **conv1_hls_0** để tăng tốc. Mô hình **tiny_cnn_v5_tail_int8.tflite** giữ phần mạng sau MaxPooling1, tránh tính lại lớp đã thực hiện trong FPGA.

Phương án	Quy trình suy luận
ARM thuần	Ảnh INT8 → Tiny CNN V5 INT8 đầy đủ → 4 lớp
ARM + FPGA	Ảnh INT8 → DMA → FPGA (Conv1, ReLU, Pool1) → ARM TFLite tail → 4 lớp

Step 2. Chuẩn bị ảnh và chương trình trên PYNQ-Z2

Sử dụng ảnh MRI **brisc2025_test_00023_gl_ax_t1.jpg**. Chương trình chuyển ảnh sang grayscale, nội suy bilinear về 96 × 96 và lượng tử hóa thành INT8 theo phép **pixel − 128**. Tensor đầu vào có 9.216 byte; tensor sau Pool1 kích thước 47 × 47 × 8, tương ứng 17.672 byte.

Các tệp cần có trong thư mục Notebook **Lab_05**: conv1_accel.bit, conv1_accel.hwh, tiny_cnn_v5_int8.tflite, tiny_cnn_v5_tail_int8.tflite và ảnh MRI thử nghiệm. Chương trình dùng TFLite Runtime trên ARM và thư viện PYNQ điều khiển Overlay, DMA.

Step 3. Mã nguồn Notebook Lab_05_Conv1_Test.ipynb

Mã nguồn dưới đây là phiên bản **cuối cùng đã chạy thành công**. Chương trình warm-up 5 lượt, đo 30 lượt; thay đổi thứ tự thực thi ARM và Hybrid sau mỗi lượt, kiểm tra đầu ra INT8 bằng np.array_equal và tính trung vị thời gian. Không đưa các mã thử nghiệm hoặc sửa lỗi trung gian.

```
# LAB 05 - MRI CNN ON PYNQ-Z2
# ARM only vs ARM + FPGA
# FPGA: Conv1 + ReLU + MaxPool1
# ARM : Remaining CNN layers

import time
import numpy as np
from PIL import Image
from pynq import Overlay, allocate
from tflite_runtime.interpreter import Interpreter

IMAGE = "brisc2025_test_00023_gl_ax_t1.jpg"
RUNS = 30
WARMUP = 5

# 1. Prepare MRI image (INT8, 96x96)
img = Image.open(IMAGE).convert("L")
img = img.resize((96, 96), Image.BILINEAR)
x = (np.asarray(img, dtype=np.int16) - 128)
x = x.astype(np.int8).reshape(1, 96, 96, 1)

# 2. Load FPGA and TFLite models
ol = Overlay("conv1_accel.bit")
dma = ol.axi_dma_0
conv1 = ol.conv1_hls_0

full = Interpreter(
```

```
    model_path="tiny_cnn_v5_int8.tflite",
    num_threads=1
)
tail = Interpreter(
    model_path="tiny_cnn_v5_tail_int8.tflite",
    num_threads=1
)

full.allocate_tensors()
tail.allocate_tensors()

fi = full.get_input_details()[0]["index"]
fo = full.get_output_details()[0]["index"]
ti = tail.get_input_details()[0]["index"]
to = tail.get_output_details()[0]["index"]

ibuf = allocate(shape=(9216,), dtype=np.int8)
obuf = allocate(shape=(47, 47, 8), dtype=np.int8)

# 3. Entire CNN on ARM
def run_arm():
    t0 = time.perf_counter()

    full.set_tensor(fi, x)
    full.invoke()
    result = full.get_tensor(fo)
```

```
    ms = (time.perf_counter() - t0) * 1000
    return result, ms

# 4. FPGA Conv1 + ARM remaining CNN
def run_hybrid():
    t0 = time.perf_counter()

    ibuf[:] = x.reshape(-1)
    ibuf.flush()

    dma.recvchannel.transfer(obuf)
    conv1.write(0x00, 1)
    dma.sendchannel.transfer(ibuf)

    # Poll DMA without artificial sleep
    deadline = time.monotonic() + 3.0

    while True:
        tx = dma.mmio.read(0x04)
        rx = dma.mmio.read(0x34)

        if (tx | rx) & 0x70:
            raise RuntimeError(
                f"DMA error: TX={hex(tx)}, RX={hex(rx)}"
            )

        if (tx & 2) and (rx & 2):
```

```
            break

        if time.monotonic() >= deadline:
            raise TimeoutError("DMA timeout")

    obuf.invalidate()
    t_fpga = time.perf_counter()

    pool1 = obuf.reshape(1, 47, 47, 8)
    tail.set_tensor(ti, pool1)
    tail.invoke()
    result = tail.get_tensor(to)

    t_end = time.perf_counter()

    return (
        result,
        (t_fpga - t0) * 1000,
        (t_end - t_fpga) * 1000,
        (t_end - t0) * 1000
    )

try:
    # 5. Warm-up
    print("Warming up...")

    for _ in range(WARMUP):
```

```
        a, _ = run_arm()
        b, _, _, _ = run_hybrid()
        assert np.array_equal(a, b), "Output mismatch"

    # 6. Benchmark with alternating execution order
    arm_times = []
    fpga_times = []
    tail_times = []
    hybrid_times = []

    print(f"Benchmarking {RUNS} runs...")

    for i in range(RUNS):
        if i % 2 == 0:
            a, arm_ms = run_arm()
            b, fpga_ms, tail_ms, hybrid_ms = run_hybrid()
        else:
            b, fpga_ms, tail_ms, hybrid_ms = run_hybrid()
            a, arm_ms = run_arm()

        if not np.array_equal(a, b):
            raise RuntimeError(f"Mismatch at run {i+1}")

        arm_times.append(arm_ms)
        fpga_times.append(fpga_ms)
        tail_times.append(tail_ms)
        hybrid_times.append(hybrid_ms)
```

```
    # 7. Results (median)
    arm_ms = np.median(arm_times)
    fpga_ms = np.median(fpga_times)
    tail_ms = np.median(tail_times)
    hybrid_ms = np.median(hybrid_times)

    scale, zp = tail.get_output_details()[0]["quantization"]
    scores = (b[0].astype(np.float32) - zp) * scale

    print("\n=== MRI CLASSIFICATION ===")
    print("Image:", IMAGE)
    print("Predicted class index:", int(np.argmax(scores)))
    print("Scores:", np.round(scores, 4))
    print("ARM output:", a.reshape(-1))
    print("Hybrid output:", b.reshape(-1))
    print("All outputs match: True")

    print("\n=== PERFORMANCE (MEDIAN) ===")
    print("Measured runs:", RUNS)
    print(f"ARM only       : {arm_ms:.3f} ms")
    print(f"FPGA + DMA     : {fpga_ms:.3f} ms")
    print(f"ARM tail       : {tail_ms:.3f} ms")
    print(f"ARM + FPGA     : {hybrid_ms:.3f} ms")
    print(f"Speedup        : {arm_ms/hybrid_ms:.3f}x")

    print("\nPASS: Hybrid CNN verified")
```

```
finally:
    ibuf.freebuffer()
    obuf.freebuffer()
```

Step 4. Phương pháp đo và đối chiếu kết quả

Hàm **run_arm()** thực hiện toàn bộ CNN trong TFLite trên ARM. Hàm **run_hybrid()** chuyển dữ liệu qua DMA, khởi động IP Conv1 trên FPGA, nhận kết quả Pool1 rồi thực hiện các lớp còn lại bằng TFLite tail. Hai phương án sử dụng cùng ảnh, cùng tham số mô hình và cùng quy trình lượng tử hóa.

Thời gian đo bằng time.perf_counter() chỉ bao gồm suy luận và truyền dữ liệu DMA, **không bao gồm** đọc và resize ảnh, nạp Bitstream hay khởi tạo mô hình. Mỗi lượt kiểm tra toàn bộ tensor đầu ra INT8. Các số liệu là **trung vị của 30 lượt đo trên một ảnh MRI**.

Step 5. Kết quả thực nghiệm

Chương trình đã kiểm chứng đầu ra ARM và Hybrid bằng nhau, cùng nhận được [127, −128, −128, −128] và dự đoán lớp chỉ số 0. Điểm số giải lượng tử của lớp này xấp xỉ 0,9961. Thời gian suy luận đo được trên PYNQ-Z2 như bảng sau.

Chỉ tiêu	Giá trị
Số lượt warm-up	5
Số lượt đo	30
ARM thuần (trung vị)	23,626 ms
FPGA + DMA (trung vị)	1,507 ms
ARM tail (trung vị)	16,554 ms
ARM + FPGA (trung vị)	**18,055 ms**
Speedup	**1,309×**
Đầu ra ARM và Hybrid	Khớp 30/30

```
Warming up...
Benchmarking 30 runs...

=== MRI CLASSIFICATION ===
Image: brisc2025_test_00023_gl_ax_t1.j
Predicted class index: 0
Scores: [0.9961 0.     0.     0.    ]
ARM output: [ 127 -128 -128 -128]
Hybrid output: [ 127 -128 -128 -128]
All outputs match: True

=== PERFORMANCE (MEDIAN) ===
Measured runs: 30
ARM only        : 23.626 ms
FPGA + DMA      : 1.507 ms
ARM tail        : 16.554 ms
ARM + FPGA      : 18.055 ms
Speedup         : 1.309x

PASS: Hybrid CNN verified
```

Hình 5.1. Kết quả phân loại ảnh MRI và benchmark 30 lượt trên PYNQ-Z2.

Từ số liệu trung vị, hệ thống kết hợp giảm 5,571 ms/ảnh so với ARM thuần, tương đương **tăng tốc 1,309×**. Đây là thời gian phần suy luận kết hợp DMA và TFLite tail, không phải riêng thời gian tính toán bên trong FPGA.

Step 6. Nhận xét và kết luận

Lab 05 chứng minh khả năng dùng FPGA để tăng tốc Conv1 + ReLU + MaxPooling1 trong mô hình Tiny CNN V5 INT8, đồng thời bảo toàn kết quả suy luận so với ARM thuần trên ảnh thử nghiệm. Phần tăng tốc này được kiểm chứng tiếp trên toàn bộ 995 ảnh Test tại Lab 06. Không dùng score 0,9961 của một ảnh để suy ra độ chính xác trên tập dữ liệu hoặc độ tin cậy chẩn đoán y khoa.